

FedFlux: Federated Adaptive Global Pruning via Network Flux

Anonymous Authors¹

Abstract

Federated dynamic pruning has emerged as a promising solution to deploy federated learning (FL) on resource-constrained devices by bypassing the memory-intensive dense training phase. However, current federated dynamic pruning frameworks suffer from two fundamental limitations: (1) **Rigidity of Layer-wise Sparsity**, arising from static layer-wise sparsity ratios that fail to adapt to shifting layer importance; and (2) **Structural Instability**, caused by magnitude-based pruning on "blurred" global aggregation. In this paper, we propose **FedFlux**, a novel framework that reformulates structural optimization through the lens of **Network Flux**. We demonstrate that the established Synaptic Flow (SynFlow) criterion is a restricted special case of our generalized Network Flux paradigm. Building on this theoretical foundation, we introduce **Adaptive Flux Redistribution (AFR)**, a data-free mechanism that empowers the server to dynamically re-partition the global sparsity budget. With AFR, FedFlux adaptively evolves the model topology to match the information demand of each layer. Extensive experiments across multiple benchmarks demonstrate that FedFlux significantly outperforms state-of-the-art (SOTA) federated pruning methods, particularly under extreme resource constraints. For example, on CIFAR-10 with a ViT in 90% sparsity, FedFlux achieves 6% higher accuracy than the SOTA baseline. The code is available at <https://anonymous.4open.science/r/FedFlux-23C1/>.

1. Introduction

Federated Learning (FL) (McMahan et al., 2017) has emerged as a pivotal paradigm for privacy-preserving col-

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

laborative intelligence, enabling decentralized edge devices to train global models without exposing sensitive local data. However, the deployment of state-of-the-art deep neural networks (DNNs) in FL is fundamentally constrained by the high computational and communication costs associated with dense parameters, which often exceed the hardware limitations of heterogeneous edge nodes. To alleviate these overheads, federated dynamic pruning has gained significant traction (Bibikar et al., 2022; Tian et al., 2024; Jiang et al., 2022; Huang et al., 2024; 2023). By periodically reconfiguring the sparse topology during training, federated dynamic pruning avoids the prohibitive memory requirements of dense pre-training, facilitating the development of efficient models directly on resource-constrained devices.

Despite its promise, existing federated dynamic pruning frameworks are hindered by two significant theoretical and practical challenges: (1) **Rigidity of Layer-wise Sparsity**: Most current methods employ heuristic, static layer-wise sparsity ratios (e.g., Erdős-Rényi-Kernel (Evci et al., 2020)). Such configurations fail to adapt to the evolving importance of different layers throughout the training process, leading to "bottlenecked" representation learning. (2) **Structural Instability under Aggregation**: Modern federated dynamic pruning relies on server-side magnitude pruning to synchronize global structures. However, weight averaging in FL is inherently "lossy" as aggregating divergent local updates often blurs critical parameter signals. Consequently, applying magnitude-based pruning to a blurred global aggregate risks discarding vital connections whose signals are suppressed by the averaging process, resulting in suboptimal convergence.

In this paper, we propose **FedFlux**, a novel federated sparse training framework that overcomes these limitations by introducing the concept of **Network Flux**. We posit that the structural importance of a network should not be viewed as a static property, but as a dynamic flow of information capacity across the architecture. Crucially, we demonstrate that the widely-used Synaptic Flow (SynFlow) (Tanaka et al., 2020) is merely a restricted, first-order special case of our generalized Network Flux. Unlike conventional methods that prune weights within fixed layer-wise constraints, our proposed **Adaptive Flux Reallocation (AFR)** mechanism empowers the server to dynamically re-partition the global parameter budget. By analyzing the second-order Network

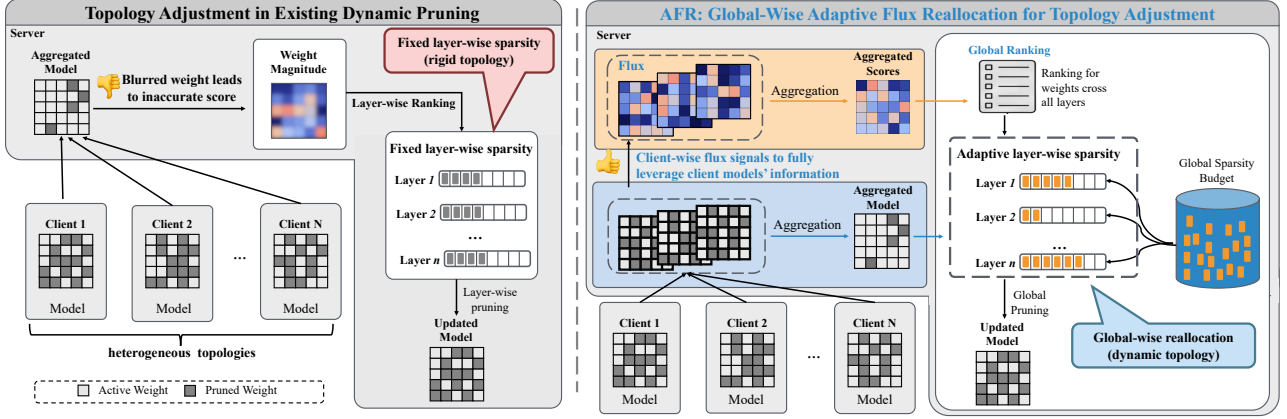


Figure 1. Overview of topology adjustment in existing federated dynamic pruning and AFR. (Left): existing methods apply magnitude pruning on aggregated (blurred) weights and enforce fixed layer-wise sparsity. (Right): AFR in FedFlux aggregates client-wise **Flux** scores and performs global-wise pruning with a global budget to reallocate sparsity across layers.

Flux, the server identifies layers with high information capacity and reallocates parameters accordingly, effectively evolving the network topology to match the complexity of the federated task.

By operating in the “Flux” domain rather than the “Weight” domain, FedFlux addresses the instability of server-side re-configuration. Instead of pruning a blurred global weight aggregate, FedFlux evaluates structural integrity through the lens of Network Flux preservation, ensuring that critical parameter pathways are maintained even in the presence of highly heterogeneous local updates. Experimental results across various benchmarks demonstrate that FedFlux significantly outperforms state-of-the-art federated pruning methods in both accuracy and communication efficiency, particularly in non-IID and resource-constrained scenarios.

To evaluate the effectiveness of FedFlux, we conduct extensive experiments across diverse Computer Vision (CV) and Natural Language Processing (NLP) benchmarks, comparing our approach to a suite of state-of-the-art (SOTA) federated pruning baselines. Our results demonstrate that FedFlux achieves a superior Pareto frontier between model performance and resource efficiency. Notably, in challenging non-IID scenarios under a 90% sparsity constraint, FedFlux achieves accuracy gains of over 6% compared to SOTA baseline methods on the CIFAR-10 dataset using a Vision Transformer (ViT) (Han et al., 2022) model.

2. Preliminary and Limitations

2.1. Federated Dynamic Pruning

Federated dynamic pruning aims to collaboratively optimize a global sparse model while minimizing on-device memory and communication overhead. The model is represented by $W \odot m$, where $W \in \mathbb{R}^N$ denotes the model weights and

$m \in \{0, 1\}^N$ is a binary mask. The objective is to minimize the global loss $F(\cdot)$:

$$\min_{W, m} F(W \odot m) = \sum_{k=1}^K p_k F_k(W \odot m, \mathcal{D}_k), \quad \text{s.t.} \quad \frac{\|m\|_0}{N} = 1 - s, \quad (1)$$

where $\mathcal{D}_k$ is the private dataset of client k , $p_k = \frac{|\mathcal{D}_k|}{\sum_j |\mathcal{D}_j|}$ is the aggregation weight, and s is the target sparsity. In each communication round t , the server broadcasts the global model (W_t, m_t) . Participating clients initialize their local models $W_t^k \odot m_t^k$ and perform local sparse training:

$$W_{t+1}^k \odot m_t^k = W_t^k \odot m_t^k - \eta_t \nabla F_k(W_t^k \odot m_t^k, \mathcal{D}_k) \odot m_t^k, \quad (2)$$

where η_t is the learning rate. Following training, the model topology is typically adjusted to adapt to the underlying data distribution.

Some approaches (Jiang et al., 2022; Huang et al., 2023; 2025) adjust the model sparse topology on the server by requiring clients to upload local gradients ∇F_k . However, such strategies introduce significant extra communication costs due to the transmission of dense gradient information or related metadata from the clients to the server.

To mitigate these communication overheads, other frameworks (Bibikar et al., 2022; Tian et al., 2024) perform topology adjustments locally on the clients. These methods adjust the local topology m^k via layer-wise pruning and growing for each layer l based on weight and gradient magnitudes:

$$m_{t+\frac{1}{2}}^{k,(l)} = \text{Prune\&Grow}(m_t^{k,(l)}, |W_{t+1}^{k,(l)}|, |\nabla F_k^{(l)}|), \quad (3)$$

where the number of connections for pruning and growing is equal. After that, as shown in Fig. 1(left), the server then performs a weighted aggregation of the local weights and a

union of the local topologies:

$$W_{t+1} = \sum_{k=1}^K p_k W_{t+1}^k, \quad m_{t+\frac{1}{2}} = \bigcup_{k=1}^K m_{t+\frac{1}{2}}^k. \quad (4)$$

Because the union mask $m_{t+\frac{1}{2}}$ is generally denser than the target sparsity s , a further pruning step is applied to the aggregated model at the server to restore the global sparsity s for the subsequent round:

$$m_{t+1}^{(l)} = \text{Prune}(m_{t+\frac{1}{2}}^{(l)}, |W_{t+1}^{(l)}|, s^{(l)}), \quad (5)$$

where the $\text{Prune}(\cdot)$ operation is executed via layer-wise ranking for each layer l based on the predefined layer-wise sparsity ratio $s^{(l)}$. Further details regarding the federated dynamic pruning process are provided in Appendix D.

2.2. Limitations in Current Approaches

Despite the effectiveness of Federated dynamic pruning in reducing overhead, current methodologies suffer from two primary limitations that impede the discovery of optimal subnetwork topologies.

Rigidity of Layer-wise Sparsity: Most existing federated dynamic pruning frameworks rely on a predefined layer-wise sparsity ratio $s^{(l)}$, such as the Erdős-Rényi-Kernel, which remains **fixed** throughout the entire training process. By performing topology adjustments via layer-wise ranking, these methods implicitly assume that *the importance distribution across layers is static and known a priori*. This rigidity prevents the model from dynamically reallocating its parameter budget across the global architecture. Consequently, layers that may require higher capacity to capture complex features are constrained by their initial sparsity assignments, while less critical layers may retain unnecessary parameters, leading to a sub-optimal global topology.

Structural Instability: The standard practice of performing the further pruning step on the aggregated weight magnitude $|W_{t+1}|$, as shown in Eq. 5, introduces significant structural instability. It assumes that *the averaged weight values are a reliable proxy for parameter importance across all clients*. However, due to the non-IID nature of federated learning, a parameter that is critical for a specific subset of clients may have a small magnitude in the aggregated global model. Pruning based solely on the aggregated model risks discarding these locally significant features, leading to a "mask collapse" where the global topology fails to represent the diverse feature spaces of the participating individuals.

3. Methodology

3.1. SynFlow vs. Network Flux

To address the **Rigidity of Layer-wise Sparsity** identified in Sec. 2.2, we advocate for a global pruning mechanism that

evolves the network topology from a holistic, cross-layer perspective:

$$m_{t+1} = \text{Prune}(m_{t+\frac{1}{2}}, \mathcal{S}(W_{t+1}), s), \quad (6)$$

where $\mathcal{S}(W_{t+1})$ denotes the importance score for the weights. Unlike layer-wise constraints, global pruning utilizes a single budget s to select parameters across all layers jointly. Consequently, the effective sparsity ratio of each layer is no longer static but adjusts implicitly according to the global ranking induced by $\mathcal{S}(\cdot)$. This flexibility is particularly advantageous in federated learning, where heterogeneous client updates can shift the relative importance and scale of different layers across communication rounds.

Defining an effective score $\mathcal{S}(W)$ under the constraints of data privacy and resource limitations is non-trivial. Data-free criteria are preferred to avoid privacy leakage and local overhead. While weight magnitude is a common heuristic, magnitude-based scores are often not theoretically comparable across different layers and are prone to inducing layer collapse. SynFlow (Tanaka et al., 2020) currently serves as a SOTA data-free pruning metric by utilizing the concept of information flow. It defines a flow objective $\Phi(W)$ as:

$$\Phi(W) = \mathbf{1}^\top f_{|W|}(\mathbf{1}), \quad (7)$$

where $f_{|W|}(\cdot)$ represents the forward pass with weights replaced by their absolute values. The importance of each weight is then measured as:

$$\mathcal{S}(W_i) = \frac{\partial \Phi(W)}{\partial W_i} W_i. \quad (8)$$

Despite its success at initialization, SynFlow’s performance is limited in dynamic pruning scenarios, as shown in Fig. 7. We posit that this gap arises because **SynFlow is merely a first-order approximation of the true change in network capacity**. By defining the pruning of weight W_i as a perturbation $-W_i e_i$, where e_i is the unit vector whose i -th element is 1, the resulting loss in output volume, which we term the **Network Flux**, can be analyzed via Taylor expansion:

$$\Delta \Phi(W) = \Phi(W) - \Phi(W - W_i e_i) \approx \frac{\partial \Phi(W)}{\partial W_i} W_i - \frac{1}{2} W_i^2 H_{ii}(W). \quad (9)$$

While SynFlow ignores the second-order term H_{ii} , assuming the Hessian is negligible at initialization, this assumption fails during federated dynamic pruning as the model converges and weight distributions shift.

However, even the diagonal term H_{ii} is insufficient, as it treats weights in isolation. In a coupled neural network, the importance of W_i depends on its interaction with other parameters. To incorporate these interactions scalably, we move beyond the diagonal approximation. We replace

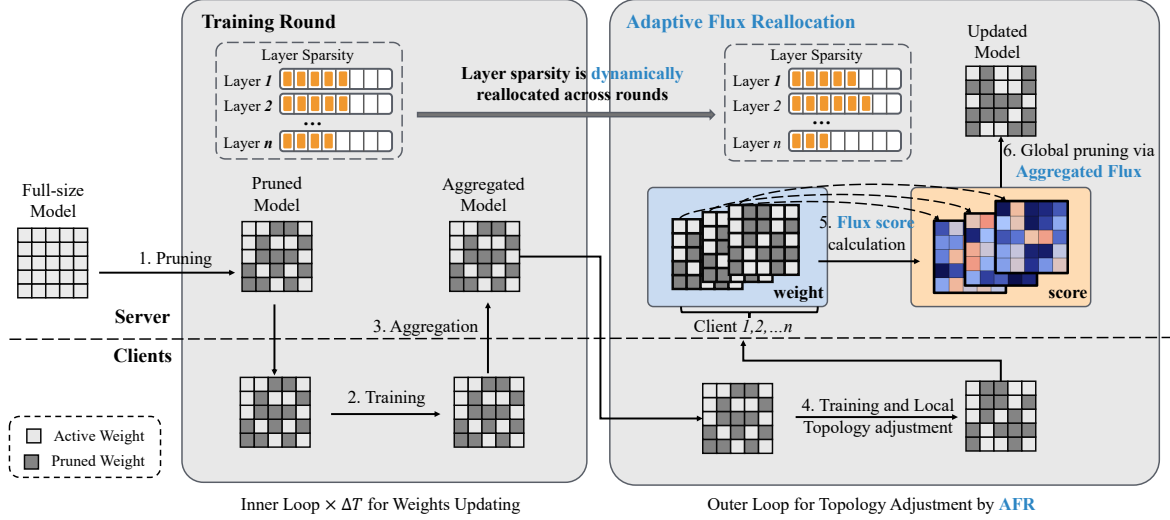


Figure 2. **Two-loop Optimization FedFlux framework with AFR.** The framework sets local weight updating on the inner loop and topology evolution on the outer loop. By aggregating Network Flux scores rather than raw weights for structural updates, FedFlux ensures stable parameter reallocation across layers even under high data heterogeneity.

the isolated term $W_i^2 H_{ii}$ with a directional curvature signal that captures the coupling between W_i and the rest of the network W : $W_i[H(W)W]_i = W_i \sum_j H_{ij} w_j$. This interaction-aware term represents the change in flux sensitivity along the current parameter direction. This quantity is efficiently computed via Pearlmutter’s trick (Loo et al., 2023) without explicit Hessian construction:

$$H(W)W = \frac{\partial}{\partial \alpha} \nabla \Phi(W + \alpha W) \Big|_{\alpha=0}, \quad (10)$$

yielding our final **Network Flux** score for $W_i \in W$ is :

$$S(W_i) \approx \underbrace{\frac{\partial \Phi(W)}{\partial W_i} W_i}_{1st \text{ Order (SynFlow)}} - \underbrace{\frac{1}{2} W_i[H(W)W]_i}_{2nd \text{ Order Interaction}}. \quad (11)$$

This formulation corrects the flux sensitivity using second-order signals that account for parameter couplings while remaining computationally tractable ($\mathcal{O}(N)$) for server-side reconfiguration.

3.2. Adaptive Flux Reallocation (AFR)

While the Network Flux provides a robust metric for cross-layer importance, applying it directly to a globally averaged model still faces the **Structural Instability** challenge identified in Sec. 2.2. Specifically, federated averaging (FedAvg) often acts as a low-pass filter; aggregating divergent updates from heterogeneous clients can *blur* high-frequency parameter signals. Performing magnitude-based or flux-based pruning on this blurred aggregate risks discarding vital connections that were significant to specific clients but suppressed by the averaging process.

To mitigate this, we propose **Adaptive Flux Reallocation (AFR)**, which decouples structural *importance estimation* from *model aggregation*. Rather than scoring a single “blurred” global weight aggregate W_t , we evaluate the Network Flux for each individual client model $\{W_t^k\}_{k=1}^K$ uploaded to the server:

$$S_{global} = \sum p_k \mathcal{S}(W_t^k), \quad k = 1, \dots, K, \quad (12)$$

where $\mathcal{S}(\cdot)$ is the second-order flux score in Eq. 11. By aggregating in the *Flux domain* rather than the *Weight domain*, AFR preserves the structural integrity of specialized features learned by local clients. This consensus map S_{global} is then used to perform global-wise pruning across all layers simultaneously, allowing the network to naturally evolve toward a more efficient sparsity distribution that matches the complexity of the federated task.

To further ensure stability, we observe that the importance of a parameter is conditional on the current topology. A one-shot global pruning decision based on a stale configuration may lead to suboptimal convergence. Therefore, we implement **Progressive Flux Pruning**. Instead of reaching the target sparsity s in a single step, the server performs n iterative sub-steps of scoring and pruning within a single communication round. This allows the flux to redistribute across the remaining connections, preventing the sudden collapse of high-capacity layers. Crucially, as all evaluations are performed server-side on uploaded models, AFR introduces *zero* additional computation or communication overhead for the resource-constrained clients. The complete procedure is detailed in Algorithm 1.

In summary, **AFR** enables server-side *global-wise* pruning by combining a *data-free* flux-based importance score with

Algorithm 1 AFR: Progressive Global Pruning with Stable Score Aggregation

Input: client weights $\{W_{t+1}^k\}_{k=1}^K$, client masks $\{m_{t+\frac{1}{2}}^k\}_{k=1}^K$, target sparsity s , pruning steps n

Output: global weights W_{t+1} , updated mask m_{t+1}

```

1:  $W_{t+1} \leftarrow \sum_{k=1}^K p_k W_{t+1}^k$ 
2:  $m_{t+\frac{1}{2}} \leftarrow \bigcup_{k=1}^K m_{t+\frac{1}{2}}^k$  // union mask
3:  $m^{(0)} \leftarrow m_{t+\frac{1}{2}}$  // mask before pruning
4:  $s^{(0)} \leftarrow 1 - \frac{\|m^{(0)}\|_0}{N}$  // sparsity before pruning
5: for iterative step  $j = 1$  to  $n$  do
6:   for client  $k = 1$  to  $K$  do
7:      $W_k^{(j)} \leftarrow W_{t+1}^k \odot m^{(j-1)}$ 
8:      $\Phi_k \leftarrow \mathbf{1}^\top f_{|W_k^{(j)}|}(\mathbf{1})$ 
9:     Calculate  $\mathcal{S}(W_k^{(j)})$  via Eq. 11
10:   end for
11:    $S_{global} \leftarrow \sum_{k=1}^K p_k \mathcal{S}(W_k^{(j)})$  // averaged flux
12:    $s^{(j)} = s^{(0)} + \frac{j}{n}(s - s^{(0)})$  // update sparsity
13:    $m^{(j)} \leftarrow \text{Prune}(m^{(j-1)}, S_{global}, s^{(j)})$ 
14: end for
15:  $m_{t+1} \leftarrow m^{(n)}$ 
16:  $W_{t+1} \leftarrow W_{t+1} \odot m_{t+1}$ 
17: Return  $(W_{t+1}, m_{t+1})$ 

```

a *stable* client-wise score aggregation strategy, yielding reliable topology refinement under heterogeneous updates without requiring any training data at the server.

3.3. The FedFlux Framework

FedFlux is a novel federated pruning framework designed to resolve the structural instability and rigidity inherent in existing decentralized dynamic pruning methods. As illustrated in Fig. 2, the framework is organized as a bi-level optimization structure that decouples client-side weight convergence from server-side topology evolution.

In the inner loop for weight updating, the server distributes the current sparse global model (W_t, m_t) to a subset of participating clients. Each client performs local sparse training to optimize weights within the fixed constraints of the global topology according to Eq. 2. By keeping the mask m_t static during the inner loop, we ensure that local gradients are focused on weight convergence rather than premature structural changes.

The core innovation of FedFlux resides in the outer loop for topology evolution. To facilitate local exploration, clients perform localized pruning and regrowth as defined in Eq. 3, which produces a set of heterogeneous local topologies tailored to their respective data distributions. Following the aggregation of weights and topology as in Eq. 4, the server leverages AFR to adaptively reconfigure the global sparse

topology in a data-free manner by aggregating client-wise model’s flux scores. This approach ensures that global-wise pruning and parameter reallocation are driven by preserved information capacity rather than blurred weight magnitudes.

By iteratively alternating between local weight optimization and topology evolution, FedFlux enables the global model to naturally flow toward an optimal layer-wise sparsity distribution. This progressive refinement ensures a smooth sparsity evolution and a more resilient global architecture, particularly in the high-sparse region. Ultimately, FedFlux provides a principled mechanism for adaptive sparse training that maintains high representation capacity while strictly adhering to resource and privacy constraints. The complete procedure of FedFlux is summarized in Algorithm 2.

3.4. Overhead Analysis

Server-side Computation Overhead. The additional overhead of FedFlux is only server-side, incurred by AFR, which evaluates the Network Flux score on the uploaded client models and then performs global-wise pruning. Let K denote the number of sampled (uploaded) clients in round t , and let N denote the number of model parameters.

(1) *Standard aggregation.* The server still performs FedAvg as usual, with time complexity $\mathcal{O}(KN)$.

(2) *AFR scoring and score aggregation.* AFR adds a scoring stage on the server: for each uploaded client model, it runs a small constant number of autodiff operations to compute the first-order SynFlow term and the interaction-aware second-order term (via an efficient Hessian–vector product), and then aggregates client-wise scores. Each evaluation is linear in parameters, so the total cost is $\mathcal{O}(KN)$ per sub-step.

(3) *Global-wise pruning / mask update.* Given the aggregated score vector, the server updates the mask via a global selection step (e.g., Top- k), which costs $\mathcal{O}(N)$

When using progressive pruning (Alg. 1), AFR repeats the prune procedure for n short sub-steps within the same round. Overall, the per-round server-side complexity can be summarized as

$$\mathcal{O}(KN) + \mathcal{O}(nKN) + \mathcal{O}(nN), \quad (13)$$

which remains in the same order as standard aggregation $\mathcal{O}(KN)$ up to a small constant factor n . **Importantly, this overhead is fully server-side and does not affect the resource-constrained clients.**

Communication overhead. FedFlux does not require clients to transmit any extra information (e.g., client gradients, flux scores, or data). AFR consumes only the client models already uploaded for aggregation, and the server returns the usual sparse global model. Therefore, the communication pattern and order remain unchanged compared

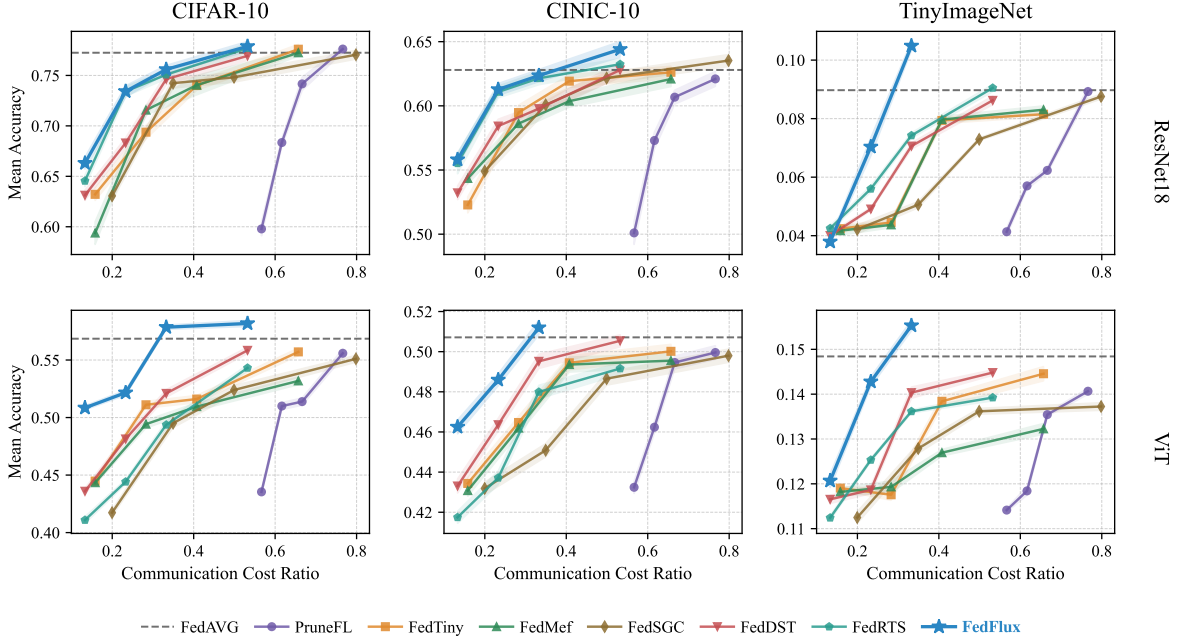


Figure 3. Testing accuracy of FedFlux and federated pruning baselines on the three datasets under different densities, where the ratio denotes communication cost relative to dense FedAvg.

with standard pruning-based FL baselines (Bibikar et al., 2022).

Practical implication. Overall, FedFlux shifts the extra cost to the server, which is typically assumed to have substantial compute in cross-device federated learning. Hence, FedFlux introduces no additional client burden and only a slight server-side overhead even for large models, which makes it practical for real-world deployment.

4. Evaluation

To assess the effectiveness of **FedFlux**, we conduct extensive experimental evaluations in this section. Each experiment is repeated five times, and we report the average results together with the corresponding standard errors. In all figures, the shaded regions denote the standard errors.

4.1. Experiment Setup

An overview of the experimental setup is given in this section, with further details deferred to Appendix C.

Datasets and Models. Following previous work (Huang et al., 2023; 2024; 2025; Bibikar et al., 2022; Jiang et al., 2022) on federated dynamic pruning, we conduct experiments on computer vision tasks using two representative models: ResNet18 (He et al., 2016) and Vision Transformer (ViT) (Han et al., 2022). Experiments are carried out on three widely used image classification datasets: CIFAR-

10 (Krizhevsky et al., 2009), CINIC-10 (Darlow et al., 2018), and TinyImageNet (Deng et al., 2009). For NLP tasks, we adopt the GPT-2-32M model on the TinyStories dataset (Elidan & Li, 2023), a language understanding benchmark designed for small-scale language models.

Federated Learning Settings. We follow the experimental protocol adopted in prior related work (Huang et al., 2023; 2024; 2025; Bibikar et al., 2022). Specifically, the training process spans $T_{\max} = 500$ communication rounds, with each client performing 5 local epochs per round. The batch size is set to 64 for computer vision tasks and 16 for natural language processing tasks. At each communication round, 10 clients are uniformly sampled from a pool of $K = 100$ clients.

To induce data heterogeneity across clients, we employ a Dirichlet distribution with concentration parameter α . The heterogeneity settings differ between CV and NLP tasks. For CV experiments, label distribution skew is introduced with $\alpha = 0.5$ by default. For NLP experiments with unlabeled data, heterogeneity is modeled through data quantity imbalance, using $\alpha = 5$ by default.

Topology adjustment is performed via an outer loop executed every $\Delta T = 10$ inner loops, and this process continues until the total number of iterations reaches $T_{\text{end}} = 300$. We adopt the Adam optimizer with a learning rate of $\eta = 0.001$.

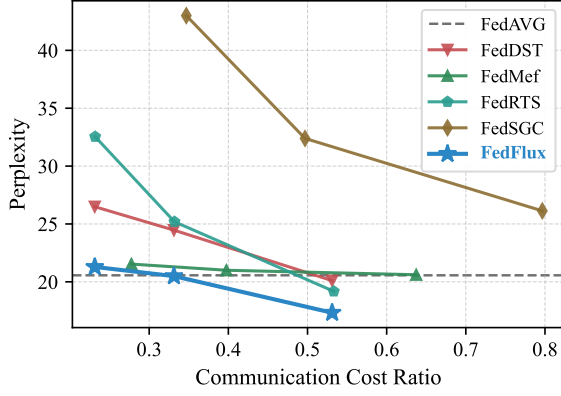


Figure 4. Test perplexity (PPL; lower is better) on TinyStories with GPT-2-32M under different target densities

Baseline Settings. We select several state-of-the-art federated pruning frameworks as baselines, including PruneFL (Jiang et al., 2022), FedTiny (Huang et al., 2023), FedDST (Bibikar et al., 2022), FedSGC (Tian et al., 2024), FedMef (Huang et al., 2024) and FedRTS (Huang et al., 2025). These methods integrate dynamic pruning into federated learning and have demonstrated strong performance across a variety of experimental settings, with further details provided in Sec. C.2.

Implementation Details. Consistent with prior studies (Huang et al., 2023; 2024), bias terms, normalization layers, and the output layer are exempted from pruning, while the overall model sparsity is equal s . The computation of training FLOPs and communication costs follows the methodology detailed in Sec. C.4.

4.2. Performance Evaluation

To demonstrate the effectiveness of FedFlux, we conduct experiments on both computer vision and natural language processing tasks under a range of target sparsity levels. Below are the main findings from our evaluations.

Computer Vision Tasks. In computer vision tasks, we evaluate FedFlux using ResNet18 and ViT models under four target sparsity levels, i.e., 50%, 70%, 80%, and 90%, on the CIFAR-10, CINIC-10, and TinyImageNet datasets. We report both classification accuracy and communication cost ratio to assess the performance–efficiency trade-off across different sparsity regimes. As shown in Fig. 3, FedFlux consistently achieves superior or competitive accuracy compared to existing federated pruning baselines at all sparsity levels. Moreover, FedFlux also delivers strong performance on transformer-based architectures; for example, on CIFAR-10 with ViT at a target sparsity of 90%, it outperforms the best competing method by more than 5 percentage points, demonstrating robust effectiveness across diverse model

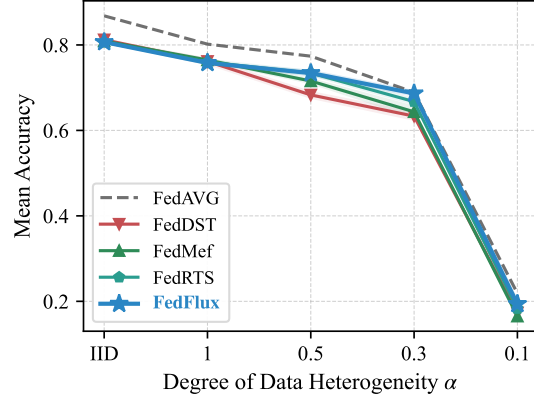


Figure 5. Accuracy on CIFAR-10 under various degrees of data heterogeneity (Dirichlet partition with different α).

families. These observed gains can be attributed to FedFlux’s adaptive sparsity evolution and its robust handling of server-side reconfiguration under federated learning scenarios.

Natural Language Processing Tasks. We further evaluate FedFlux on an NLP task using GPT-2-32M trained on TinyStories under three target sparsity levels (50%, 30%, and 20%). We report test perplexity (PPL; lower is better), which measures language modeling quality by accounting for the full predicted probability distribution. As shown in Fig. 4, FedFlux consistently attains lower PPL than federated pruning baselines across densities, demonstrating that our methods remain effective beyond vision tasks and under aggressive sparsity.

4.3. Robustness Analysis

To demonstrate the robustness of FedFlux under heterogeneous data distributions and partial client participation, we conduct experiments with varying degrees of data heterogeneity and client availability in this section.

The Impact of Data Heterogeneity.

Data heterogeneity poses a fundamental challenge in federated learning, as skewed client data distributions often impair global model aggregation. To examine how FedFlux behaves under different heterogeneity levels, we vary the Dirichlet concentration parameter α when partitioning the CIFAR-10 dataset, where smaller values of α correspond to more severe non-IID conditions. As illustrated in Fig. 5, FedFlux maintains stable performance across the entire heterogeneity spectrum and exhibits a noticeably slower degradation as data distributions become increasingly skewed. In particular, under highly heterogeneous settings, FedFlux consistently matches or surpasses competing baselines, indicating its ability to preserve effective global representations even when client data is strongly imbalanced.

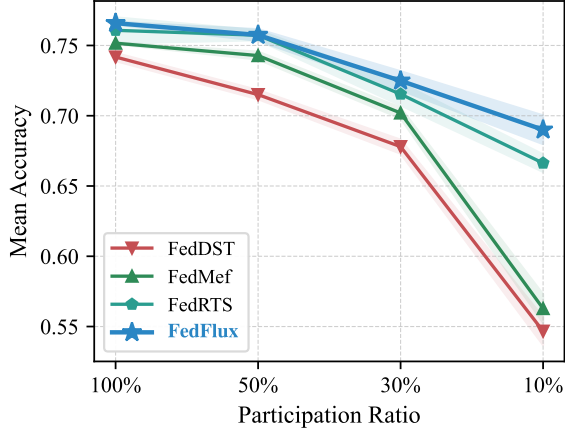


Figure 6. Accuracy on CIFAR-10 under different client participation ratios (relative to full participation).

The Impact of Client Availability.

Limited client participation is another practical challenge in federated learning, as fewer participating clients per round can lead to unstable aggregation and degraded convergence. To assess the robustness of FedFlux under partial participation, we vary the client participation ratio from 100% to 10% while fixing the total number of clients to 10. As shown in Fig. 6, FedFlux exhibits strong resilience to reduced client availability and maintains favorable performance even when only a small subset of clients participates in each communication round, which shows the advantage of FedFlux’s adaptive topology adjustment in effectively leveraging the limited client updates to refine the global model structure.

4.4. Ablation Study

To analyze the individual contributions of FedFlux, we conduct an ablation study on CIFAR-10 by varying (i) the *scoring pipeline* for score computation, Aggregate-then-Score (ATS) vs. Score-then-Aggregate (STA), and (ii) the Flux order, first-order (1st) vs. second-order (2nd).

Concretely, we compare the following variants: (1) *FedDST*; (2) *ATS-1st*, which computes first-order scores on the aggregated (blurred) global weights; (3) *FedFlux (ATS-2nd)*, which computes second-order Flux scores on the aggregated weights; (4) *FedFlux (STA-1st)*, which computes first-order scores on each client model and aggregates the scores; and (5) *FedFlux*, which aggregates client-wise second-order Flux scores. As shown in Fig. 7, second-order scoring consistently improves over first-order scoring, and STA outperforms ATS by preserving non-blurred local structural signals before aggregation. Combining both STA and 2nd-order Flux, i.e., our full **FedFlux**, achieves the best overall performance.

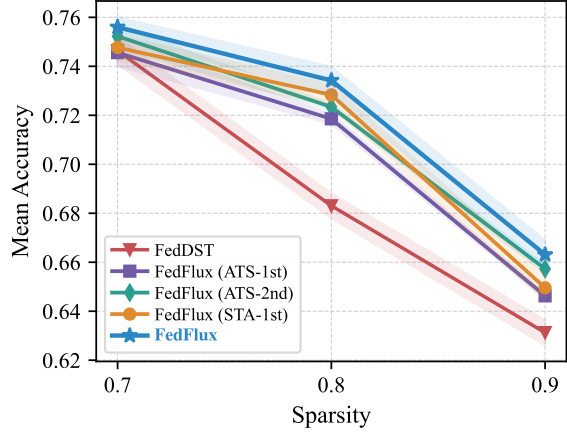


Figure 7. Ablation results on CIFAR-10 comparing Aggregate-then-Score (ATS) vs. Score-then-Aggregate (STA) and first- vs. second-order Flux scoring.

5. Conclusion

In this paper, we introduced **FedFlux**, a novel federated sparse training framework designed to address the fundamental limitations of rigidity and structural instability in existing federated dynamic pruning methods. By shifting the pruning paradigm from the traditional *weight magnitude domain* to the **Network Flux domain**, we provide a more resilient foundation for topology evolution. We theoretically demonstrated that the widely-used SynFlow is a restricted first-order approximation of our generalized second-order Network Flux, which captures critical parameter interactions via Hessian-vector products.

Our proposed **Adaptive Flux Reallocation (AFR)** mechanism enables the server to dynamically re-partition the parameter budget across layers, overcoming the bottlenecks of static layer-wise sparsity. Crucially, by aggregating flux sensitivities rather than “blurred” weight magnitudes, FedFlux maintains structural integrity even under extreme data heterogeneity. Empirical results across CV and NLP benchmarks validate the effectiveness of our approach; notably, FedFlux achieves a superior Pareto frontier between accuracy and efficiency, outperforming SOTA baselines by over 6% at 90% sparsity. We believe that the concept of Network Flux opens new avenues for understanding and optimizing sparse architectures in decentralized environments, providing a scalable path toward efficient and robust collaborative intelligence.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Bibikar, S., Vikalo, H., Wang, Z., and Chen, X. Federated dynamic sparse training: Computing less, communicating less, yet learning better. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pp. 6080–6088, 2022.
- Darlow, L. N., Crowley, E. J., Antoniou, A., and Storkey, A. J. Cinic-10 is not imagenet or cifar-10. *arXiv preprint arXiv:1810.03505*, 2018.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pp. 248–255. Ieee, 2009.
- Eldan, R. and Li, Y. Tinstories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.
- Evci, U., Gale, T., Menick, J., Castro, P. S., and Elsen, E. Rigging the lottery: Making all tickets winners. In *International Conference on Machine Learning*, pp. 2943–2952. PMLR, 2020.
- Han, K., Wang, Y., Chen, H., Chen, X., Guo, J., Liu, Z., Tang, Y., Xiao, A., Xu, C., Xu, Y., et al. A survey on vision transformer. *IEEE transactions on pattern analysis and machine intelligence*, 45(1):87–110, 2022.
- Han, S., Mao, H., and Dally, W. J. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778. IEEE, 2016.
- Huang, H., Zhang, L., Sun, C., Fang, R., Yuan, X., and Wu, D. Distributed pruning towards tiny neural networks in federated learning. In *2023 IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*, pp. 190–201. IEEE, 2023.
- Huang, H., Zhuang, W., Chen, C., and Lyu, L. Fedmef: Towards memory-efficient federated dynamic pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 27548–27557, 2024.
- Huang, H., Yang, H., Chen, Y., Ye, J., and Wu, D. Fedrts: Federated robust pruning via combinatorial thompson sampling. *arXiv preprint arXiv:2501.19122*, 2025.
- Jiang, Y., Wang, S., Valls, V., Ko, B. J., Lee, W.-H., Leung, K. K., and Tassiulas, L. Model pruning enables efficient federated learning on edge devices. *IEEE Transactions on Neural Networks and Learning Systems*, 34(12):10374–10386, 2022.
- Krizhevsky, A., Hinton, G., and et al. Learning multiple layers of features from tiny images. In *Proceedings of the International Conference on Machine Learning*. PMLR, 2009.
- Lee, N., Ajanthan, T., and Torr, P. Snip: Single-shot network pruning based on connection sensitivity. In *International Conference on Learning Representations*, 2018.
- Loo, N., Hasani, R., Lechner, M., and Rus, D. Dataset distillation with convexified implicit gradients. In *International Conference on Machine Learning*, pp. 22649–22674. PMLR, 2023.
- McMahan, B., Moore, E., Ramage, D., Hampson, S., and y Arcas, B. A. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pp. 1273–1282. PMLR, 2017.
- Mocanu, D. C., Mocanu, E., Stone, P., Nguyen, P. H., Gibescu, M., and Liotta, A. Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science. *Nature communications*, 9(1):1–12, 2018.
- Molchanov, P., Tyree, S., Karras, T., Aila, T., and Kautz, J. Pruning convolutional neural networks for resource efficient inference. In *5th International Conference on Learning Representations, ICLR 2017-Conference Track Proceedings*, 2019.
- Tanaka, H., Kunin, D., Yamins, D. L., and Ganguli, S. Pruning neural networks without any data by iteratively conserving synaptic flow. *Advances in Neural Information Processing Systems*, 33:6377–6389, 2020.
- Tian, C. X., Liu, Y., Li, H., Cheung, R. C., and Wang, S. Gradient-congruity guided federated sparse training. *arXiv preprint arXiv:2405.01189*, 2024.

A. Related Work

B. Related Work

Dynamic Sparse Training (DST). Traditional neural network pruning has transitioned from post-training compression (Han et al., 2015; Molchanov et al., 2019) toward pruning at initialization (Lee et al., 2018; Tanaka et al., 2020) and Dynamic Sparse Training (DST) (Mocanu et al., 2018; Evci et al., 2020). DST is uniquely suited for resource-constrained environments as it evolves the network topology during training, bypassing the memory-heavy requirements of dense pre-training. While popular methods like RigL (Evci et al., 2020) utilize gradient information for parameter regrowth, they are largely constrained by static, heuristic layer-wise sparsity ratios, such as the Erdős-Rényi-Kernel (ERK). FedFlux diverges from this paradigm by introducing a global, second-order Network Flux criterion that enables dynamic reallocation of the parameter budget across the entire architecture, overcoming the representational bottlenecks of rigid sparsity configurations.

Federated Dynamic Pruning. Recent efforts in Federated Learning (FL) have integrated DST to reduce the communication and computational footprint on edge devices (Jiang et al., 2022; Bibikar et al., 2022). Current research generally follows two distinct trajectories based on the locus of topological reconfiguration. The first category focuses on *server-side reconfiguration*, such as FedTiny (Huang et al., 2023), FedMef (Huang et al., 2024), and FedRTS (Huang et al., 2025). These frameworks often require clients to transmit importance scores for regrowth, potentially incurring significant communication overhead. The second category decentralizes reconfiguration by performing *client-side pruning and regrowth*, as seen in FedDST (Bibikar et al., 2022), FedSGC (Tian et al., 2024), and FedFit. While this reduces server-side dependency, it produces highly heterogeneous local topologies that necessitate complex union aggregation and subsequent pruning to satisfy global density constraints.

Crucially, both research lines remain anchored to predefined, static layer-wise densities, adjusting parameters only within fixed local budgets. FedFlux distinguishes itself by enabling an **adaptive global sparsity evolution**. By leveraging the structural consensus of Network Flux, our framework adaptively re-partitions the parameter budget across layers, ensuring structural stability and superior convergence even under severe non-IID data distributions.

C. Additional Setup Details

C.1. Datasets

We evaluate FedFlux on both computer vision and natural language processing tasks. For CV tasks, we use ResNet18 and Vision Transformer (ViT) as representative backbones and consider three standard image classification benchmarks: CIFAR-10, CINIC-10, and TinyImageNet. For NLP tasks, we use GPT-2-32M on TinyStories.

- **CIFAR-10.** This dataset contains 50,000 training images and 10,000 test images across 10 classes. Each sample is a 32×32 RGB image.
- **CINIC-10.** CINIC-10 is an extended variant of CIFAR-10 that additionally includes images from ImageNet. It provides three equally sized splits (train/validation/test), each with 90,000 images.
- **TinyImageNet.** TinyImageNet contains 200 categories, with 500 training images, 50 validation images, and 50 test images per category. Images are resized to 64×64 pixels.
- **TinyStories.** TinyStories is a synthetic short-story corpus constructed with a restricted vocabulary for small language models. We truncate each sample to 256 tokens, using 2,120,000 stories for training and 22,000 stories for testing.

C.2. Baselines

We compare FedFlux with several representative federated pruning frameworks. These baselines mainly differ in how they initialize sparse models and how they update the sparse topology during training.

- **PruneFL.** PruneFL performs adaptive pruning using an importance criterion and relies on additional bookkeeping for each parameter during adjustment. In our implementation, we use random pruning for initialization (instead of training a dense model on a powerful device) due to practical resource constraints, and we set the per-parameter time variable t_j to 1.

- **FedDST.** FedDST applies dynamic sparse training on clients and forms the global sparse topology via greedy aggregation.
- **FedTiny.** FedTiny initializes sparsity with adaptive batch-normalization selection and updates the mask using aggregated top- K gradient information.
- **FedSGC.** FedSGC adjusts the topology on-device using gradient congruity guidance; compared to FedDST-like approaches, it requires additional information exchange.
- **FedMef.** FedMef transfers essential information from pruned parameters via budget-aware extrusion and further reduces activation memory through scaled activation pruning.
- **FedRTS.** FedRTS uses Thompson sampling to guide topology adjustment with probabilistic decisions, aiming to stabilize sparse topology evolution under heterogeneity and partial participation.

C.3. Extra Information

All experiments are executed via multi-process simulation on a workstation equipped with an Nvidia RTX 5880 GPU, an Intel Core i9 CPU, and 48 GB of RAM.

C.4. Communication and Computational Cost

D. Complete Algorithm

We present the complete pseudocode of the FedFlux framework in Algorithm 2. The algorithm follows a bi-level optimization structure: the inner loop focuses on local weight optimization with a fixed sparse topology, while the outer loop evolves the global topology via Adaptive Flux Reallocation (AFR).

In the inner loop, each participating client performs E epochs of local sparse training on its private data while keeping the mask fixed. This ensures that local updates focus on optimizing weight values rather than prematurely altering the network structure.

Following local training, clients perform a local topology adjustment step using layer-wise pruning and growing based on weight magnitude and gradient information, respectively. This allows clients to explore structural variations tailored to their local data distributions.

After receiving the updated models, the server first aggregates the weights via FedAvg and forms a union of all client masks. Since the union mask typically violates the target budget, AFR then performs *progressive* global-wise pruning: at each sub-step, the server (i) masks each uploaded client model with the current topology, (ii) computes client-wise Network Flux scores in a data-free manner, (iii) aggregates scores in the flux domain to obtain a stable consensus map, and (iv) prunes globally toward an intermediate sparsity target. After n sub-steps, the updated mask m_{t+1} restores the desired sparsity while adaptively reallocating parameters across layers.

This bi-level design enables FedFlux to balance local feature specialization with global structural coherence, achieving stable and efficient sparse model convergence under heterogeneous federated settings.

Algorithm 2 FedFlux: Federated Adaptive Global Pruning via Network Flux

Input: total rounds T , local epochs E , target sparsity s , learning rate η , pruning steps n , client set $\mathcal{K}$, participation ratio C , prune-grow ratio ζ

Output: final sparse model (W_T, m_T)

- 1: Initialize global weights W_0 and mask m_0 with sparsity s
- 2: **for** round $t = 0$ **to** $T - 1$ **do**
- 3: $\mathcal{C}_t \leftarrow$ sample $\lfloor C \cdot |\mathcal{K}| \rfloor$ clients from $\mathcal{K}$
- 4: Server broadcasts (W_t, m_t) to all clients in $\mathcal{C}_t$
- 5: **for** each client $k \in \mathcal{C}_t$ **in parallel do**
- 6: $W^k \leftarrow W_t, m^k \leftarrow m_t$ *// initialize local model*
- 7: */* Inner Loop: Local Sparse Training */*
- 8: **for** local epoch $e = 1$ **to** E **do**
- 9: **for** each mini-batch $\mathcal{B} \subset \mathcal{D}_k$ **do**
- 10: $g^k \leftarrow \nabla F_k(W^k \odot m^k, \mathcal{B})$
- 11: $W^k \leftarrow W^k - \eta \cdot g^k \odot m^k$ *// Eq. 2*
- 12: **end for**
- 13: **end for**
- 14: */* Local Topology Adjustment */*
- 15: **for** each layer l **do**
- 16: $\Delta_{\text{prune}} \leftarrow \lfloor \zeta \cdot \|m^{k,(l)}\|_0 \rfloor$
- 17: $\mathcal{I}_{\text{drop}} \leftarrow \text{BottomK}(|W^{k,(l)}| \odot m^{k,(l)}, \Delta_{\text{prune}})$
- 18: $m^{k,(l)}[\mathcal{I}_{\text{drop}}] \leftarrow 0$ *// prune by magnitude*
- 19: $\mathcal{I}_{\text{grow}} \leftarrow \text{TopK}(|g^{k,(l)}| \odot (\mathbf{1} - m^{k,(l)}), \Delta_{\text{prune}})$
- 20: $m^{k,(l)}[\mathcal{I}_{\text{grow}}] \leftarrow 1$ *// grow by gradient*
- 21: **end for**
- 22: $m_{t+\frac{1}{2}}^k \leftarrow m^k, W_{t+1}^k \leftarrow W^k$
- 23: Upload $(W_{t+1}^k, m_{t+\frac{1}{2}}^k)$ to server
- 24: **end for**
- 25: */* Server Aggregation */*
- 26: $W_{t+1} \leftarrow \sum_{k \in \mathcal{C}_t} p_k W_{t+1}^k$ *// FedAvg*
- 27: $m_{t+\frac{1}{2}} \leftarrow \bigcup_{k \in \mathcal{C}_t} m_{t+\frac{1}{2}}^k$ *// union mask*
- 28: */* Outer Loop: Adaptive Flux Reallocation */*
- 29: $m^{(0)} \leftarrow m_{t+\frac{1}{2}}$ *// mask before global pruning*
- 30: $N \leftarrow |m^{(0)}|$ *// total number of parameters*
- 31: $s^{(0)} \leftarrow 1 - \frac{\|m^{(0)}\|_0}{N}$ *// current sparsity (typically < s)*
- 32: **for** iterative step $j = 1$ **to** n **do**
- 33: **for** each client $k \in \mathcal{C}_t$ **do**
- 34: $\tilde{W}^{k,(j)} \leftarrow W_{t+1}^k \odot m^{(j-1)}$
- 35: $\Phi_k \leftarrow \mathbf{1}^\top f_{|\tilde{W}^{k,(j)}|}(\mathbf{1})$ *// data-free flux objective*
- 36: $S_k^{(j)} \leftarrow \mathcal{S}(\tilde{W}^{k,(j)})$ via Eq. 11 *// Network Flux score*
- 37: **end for**
- 38: $S_{\text{global}} \leftarrow \sum_{k \in \mathcal{C}_t} p_k S_k^{(j)}$ *// stable score aggregation*
- 39: $s^{(j)} \leftarrow s^{(0)} + \frac{j}{n}(s - s^{(0)})$ *// progressive sparsity schedule*
- 40: $m^{(j)} \leftarrow \text{Prune}(m^{(j-1)}, S_{\text{global}}, s^{(j)})$ *// global-wise pruning*
- 41: **end for**
- 42: $m_{t+1} \leftarrow m^{(n)}$
- 43: $W_{t+1} \leftarrow W_{t+1} \odot m_{t+1}$
- 44: **end for**
- 45: **Return** (W_T, m_T)
